FR

Bienvenue sur Sakata

Ce portail est un sanctuaire numérique dédié à la préservation et à la transmission de l'identité culturelle du peuple **Sakata**.

Notre objectif est d'unir nos racines ancestrales avec la splendeur numérique moderne, en offrant à la fois des archives vivantes, des sagesses intemporelles et un forum d'échange intergénérationnel.

Suivant →

> *Dans la culture Sakata, on ne se présente pas par son métier mais par sa lignée. « Je suis le fils de X, qui est le neveu de Y, du clan Z. » L'arbre généalogique n'est pas un loisir — c'est l'identité même. Numériser cela, c'est offrir aux diasporas la chance de retrouver des cousins jamais rencontrés.*

> *Mowei te akoyebe na nzela ya bankoko na ye. — « L'enfant ne se connaît qu'à travers ses ancêtres. »*
>
> — Proverbe Sakata

POURQUOI

Le besoin culturel

- **Identité par filiation** — Chez les Sakata, la lignée maternelle (matrilinéaire dans certaines branches) et paternelle structurent les héritages, les alliances de mariage, les responsabilités rituelles.
- **Mariages préventifs** — Pas de mariage entre cousins jusqu'au 4e degré. Sans connaissance précise, les diasporas risquent des unions tabou sans le savoir.
- **Reconstitution post-déplacements** — Guerres, exodes, migrations forcées ont dispersé les familles. Beaucoup cherchent activement à reconnecter.
- **Transmission** — Sans arbre tangible, chaque génération qui passe perd 30% de la mémoire familiale.

COMPARABLE EXISTANT

Geni.com, Ancestry.com sont occidentalo-centrés et ne gèrent ni la polygamie traditionnelle, ni les relations claniques (un clan ≠ une famille nucléaire), ni les noms en kisakata. C'est un trou de marché.

SCHÉMA

Modélisation des données

Pourquoi PostgreSQL et pas Neo4j ?

CHOIX

PostgreSQL avec table de relations + CTE récursives, plutôt qu'une base graphe dédiée (Neo4j, ArangoDB).

CRITÈRE	POSTGRESQL + CTE	NEO4J	VERDICT
Coût	Inclus Supabase	Hosting séparé 50-200$/mois	PostgreSQL
Profondeur typique	5-10 générations max	Illimitée	Égal à cette échelle
Performance < 10k personnes	Très bonne	Très bonne	Égal
RLS native	✅	❌ (à coder)	PostgreSQL
Intégration Sakata	Déjà là	Synchroniser deux bases	PostgreSQL

Schéma SQL

CODE SQL

```sql
-- Une personne = un nœud du graphe
CREATE TABLE genealogy_persons (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  family_tree_id UUID REFERENCES genealogy_trees(id),
  name_first TEXT NOT NULL,
  name_kisakata TEXT,                    -- nom traditionnel
  name_clan TEXT,                        -- clan Sakata
  birth_year INT,                        -- année seule (souvent inconnue)
  birth_year_estimated BOOLEAN DEFAULT FALSE,
  death_year INT,
  origin_village TEXT,
  notes TEXT,                             -- récit libre
  is_living BOOLEAN DEFAULT TRUE,
  user_id UUID REFERENCES profiles(id), -- si la personne a un compte Sakata
  created_by UUID REFERENCES profiles(id),
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Une relation = une arête typée
CREATE TABLE genealogy_relations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  family_tree_id UUID REFERENCES genealogy_trees(id),
  from_person_id UUID REFERENCES genealogy_persons(id),
  to_person_id UUID REFERENCES genealogy_persons(id),
  relation_type TEXT NOT NULL CHECK (relation_type IN (
    'parent', 'spouse', 'sibling', 'adopted_parent', 'clan_member'
  )),
  -- Pour les mariages multiples (polygamie traditionnelle)
  marriage_order INT,
  start_year INT,
  end_year INT,
  end_reason TEXT,                        -- 'death', 'divorce', 'separation'
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(from_person_id, to_person_id, relation_type, marriage_order)
);

-- Un arbre = un espace de partage
CREATE TABLE genealogy_trees (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  owner_id UUID REFERENCES profiles(id),
  name TEXT NOT NULL,
  description TEXT,
  visibility TEXT DEFAULT 'private',     -- private, family, clan, public
  share_token TEXT,                       -- pour partage par lien
```

```
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Membres autorisés (visibility = 'family')
CREATE TABLE genealogy_tree_members (
  tree_id UUID REFERENCES genealogy_trees(id),
  user_id UUID REFERENCES profiles(id),
  role TEXT DEFAULT 'viewer',            -- viewer, editor, owner
  invited_by UUID REFERENCES profiles(id),
  PRIMARY KEY (tree_id, user_id)
);
```

Requête : ancêtres jusqu'au Nᵉ degré

CODE **SQL**

```
-- Trouver tous les ancêtres d'une personne (CTE récursive)
WITH RECURSIVE ancestors AS (
  SELECT
    r.from_person_id AS ancestor_id,
    1 AS generation
  FROM genealogy_relations r
  WHERE r.to_person_id = $1 AND r.relation_type = 'parent'

  UNION ALL

  SELECT
    r.from_person_id,
    a.generation + 1
  FROM genealogy_relations r
  JOIN ancestors a ON r.to_person_id = a.ancestor_id
  WHERE r.relation_type = 'parent' AND a.generation < 10
)
SELECT p.*, a.generation
FROM ancestors a
JOIN genealogy_persons p ON p.id = a.ancestor_id
ORDER BY a.generation, p.birth_year;
```

FRONTEND

Visualisation D3.js

Pourquoi D3 et pas une lib généalogie spécifique ?

CHOIX

D3.js custom plutôt que `familytree-js`, `balkan-genealogy` ou autres libs payantes.

- Les libs spécialisées coûtent cher (200-2 000$ licence) ou sont fermées.
- Aucune ne gère bien les mariages polygames (cas Sakata fréquent).
- D3 + force-directed graph permet un layout adaptatif aux structures non-binaires.
- Contrôle total sur le style (Brume design system).

Trois layouts possibles

LAYOUT	CAS D'USAGE	LIB
Tree (vertical/horizontal)	Ascendance pure (parents → enfants)	d3.tree()
Force-directed	Réseau familial complet (mariages, clans)	d3.forceSimulation()
Sunburst radial	Vue ancêtres centrée sur ego	d3.partition() + arc

L'utilisateur peut basculer entre les trois selon ce qu'il cherche.

RLS & VISIBILITÉ

Confidentialité (sujet sensible)

DONNÉES ULTRA-SENSIBLES

Liens familiaux, dates de naissance, lieux d'origine — tout ça est exploitable pour identifier précisément un individu. RLS strictes + UI claire sur la visibilité.

Quatre niveaux de visibilité

NIVEAU	QUI VOIT	CAS D'USAGE
private	Seul l'owner du tree	Brouillon personnel
family	Members invités	Famille élargie qui collabore
clan	Tous les users avec même <code>name_clan</code>	Reconnexion clanique
public	Tout le monde	Personnages historiques, chefs reconnus

Personnes vivantes vs décédées

Pour les personnes vivantes, masquer par défaut date de naissance précise et adresse. Anonymisation possible : « Mère » / « Frère cadet » sans nom complet si l'utilisateur n'a pas donné son consentement.

EXÉCUTION

Plan d'implémentation — 5 sprints

Sprint 1 — Schéma + CRUD (1 semaine)

1. Migration SQL des 4 tables.
2. RLS pour chaque table selon les 4 niveaux de visibilité.
3. API Routes : `/api/genealogy/persons`, `/api/genealogy/relations`.
4. Tests RLS avec deux comptes utilisateurs distincts.

Sprint 2 — Éditeur (2 semaines)

1. Interface React pour ajouter/modifier des personnes.
2. Glisser-déposer pour créer des relations.
3. Validation : pas de cycle (A parent de B parent de A impossible).
4. Auto-suggestion : si l'utilisateur tape « X marié à Y », créer Y et la relation.

Sprint 3 — Visualisation D3 (2 semaines)

1. Layout tree de base (vertical, ascendance).
2. Switch vers force-directed pour vue réseau.
3. Zoom/pan, click pour focus sur une personne.
4. Photo (avatar) si fournie, initiales sinon.

Sprint 4 — Partage & invitations (1 semaine)

1. Génération de lien partage avec token.
2. Invitation par email d'un parent vers un tree.
3. Acceptation et fusion de trees (deux cousins qui réalisent qu'ils ont des ancêtres communs).

Sprint 5 — Recherche cross-trees (1 semaine)

1. Index sur `name_clan`, `origin_village`, `birth_year`.
2. Recherche fuzzy : « Personne née ~1920 dans village X » → suggestions de matches dans d'autres trees publics.
3. Validation manuelle de fusion (consentement des deux owners).

LUCIDITÉ

Risques & mitigations

RISQUE	PROBABILITÉ	IMPACT	MITIGATION
Données fausses (légendes familiales) prises pour vérité	Élevée	Moyen	Champ <code>source_certainty</code> par fait + indicateurs visuels (« mémoire orale »)
Conflits familiaux (versions divergentes)	Élevée	Haut	Permettre plusieurs trees concurrents, pas de fusion forcée
RGPD : ajouter quelqu'un sans consentement	Moyenne	Critique (légal)	Personnes vivantes anonymisées par défaut + procédure de retrait sur demande
Performance avec 5 000+ personnes	Faible (rare cas)	Moyen	Pagination des relations + virtualisation D3

KPIS

Métriques de succès

- **3 mois post-lancement :** 100 trees créés, 2 000 personnes cataloguées.
- **6 mois :** 5 fusions de trees inter-familles, 3 reconnexions diaspora documentées.
- **12 mois :** Article presse / podcast diaspora qui mentionne une « retrouvaille Sakata ».

« Moto oyo ayebi te kotombola miso na likolo, akoki te koyeba esika oyo mbula eutaka. » — Proverbe Sakata

SAKATA

© 2026 — Brume de la Rivière. Tous droits réservés.

CULTURE ECOLE LANGUE HISTOIRE FORUM